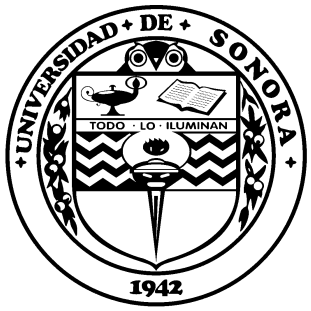


Universidad de Sonora

Licenciatura en Ciencias de la Computación

Demostración de la solución al problema Readers-Writers

Tarea individual

Materia: Sistemas Operativos
Periodo: 2026-2
Profesor: Dr. Donald José Rodríguez Úbeda

Alumno

Angel Fernando Bórquez Guerrero Expediente 219208106

Hermosillo, Sonora Noviembre de 2026

Índice

1. Introducción **2**

2. Condiciones de la solución al problema de la sección crítica **2**

3. El algoritmo y el modelo **3**
3.1. Código 3
3.2. Supuestos 3

4. Demostración **4**
4.1. Lemas 4
4.2. Exclusión mutua 5
4.3. Progreso 6
4.4. Espera limitada 7

5. Verificación con un verificador de modelos **9**

6. Conclusiones **10**

Referencias **11**

1. Introducción

El problema de los procesos Lector-Escritor (*Readers-Writers*) plantea un conjunto de procesos lectores y otro de procesos escritores que comparten un recurso, por ejemplo una base de datos. Varios lectores pueden acceder al recurso al mismo tiempo, pero un escritor necesita acceso exclusivo (Rodríguez Úbeda, 2026; Silberschatz et al., 2018).

La actividad consiste en demostrar que el algoritmo presentado en el material de la unidad (semáforos `mutex` y `rw_mutex` y el contador `readcount`) cumple las condiciones que debe satisfacer toda solución al problema de la sección crítica: exclusión mutua, progreso y espera limitada. Para cada condición se enuncia lo que significa en este problema, se demuestra a partir de invariantes y se comprueba además con un verificador de modelos que explora todas las intercalaciones posibles de un número pequeño de procesos.

Resultado principal. El algoritmo cumple la **exclusión mutua** y el **progreso**. La **espera limitada** se cumple para los lectores (suponiendo semáforos con cola FIFO), pero **no** para los escritores: pueden sufrir inanición. Esto no es un error de implementación, sino consecuencia de la variante del problema que resuelve el algoritmo (los lectores tienen prioridad), y se demuestra con un contraejemplo en la sección 4.4.

2. Condiciones de la solución al problema de la sección crítica

El material de la unidad (Rodríguez Úbeda, 2026) establece tres condiciones:

1. **Exclusión mutua:** si un proceso está ejecutando su sección crítica, los demás no pueden ejecutar las suyas.
2. **Progreso:** si ningún proceso está en su sección crítica y hay procesos que desean entrar, la selección del siguiente no puede posponerse indefinidamente.
3. **Espera limitada:** existe un límite al número de veces que se permite entrar a otros procesos después de que un proceso solicitó entrar y antes de que su solicitud sea concedida.

Interpretación para Readers-Writers

En este problema las reglas de acceso permiten que varios lectores compartan el recurso. Por eso la exclusión mutua se lee así:

- (ME1) nunca hay dos escritores a la vez en su sección crítica;
- (ME2) nunca hay un escritor y un lector a la vez en su sección crítica.

Los lectores sí pueden coincidir entre sí; esa concurrencia es parte de lo que se pide, no una violación. La sección crítica del escritor es la escritura y la del lector es la lectura. El algoritmo corresponde a la primera variante del problema: *ningún lector esperará a menos que un escritor esté accediendo al recurso.*

3. El algoritmo y el modelo

3.1. Código

Datos compartidos: `semaphore mutex = 1, rw_mutex = 1; int readcount = 0;`. Las etiquetas de cada línea se usan en las demostraciones.

```
Escritor                          Lector
W1: wait(rw_mutex);               R1: wait(mutex);
W2: /* escribe  (SC) */           R2: readcount++;
W3: signal(rw_mutex);             R3: if (readcount == 1)
                                        wait(rw_mutex);
                                  R4: signal(mutex);
                                  R5: /* lee  (SC) */
                                  R6: wait(mutex);
                                  R7: readcount--;
                                  R8: if (readcount == 0)
                                        signal(rw_mutex);
                                  R9: signal(mutex);
```

3.2. Supuestos

1. `wait` y `signal` son atómicos y se implementan como en el material: `wait` decrementa el valor y, si queda negativo, bloquea al proceso en la cola L del semáforo; `signal` incrementa el valor y, si es ≤ 0, despierta a un proceso de L.

2. Cada sección crítica termina en tiempo finito y ningún proceso falla dentro del código de entrada, de la sección crítica o de salida.

3. El planificador es justo: un proceso que puede avanzar y no está bloqueado, eventualmente ejecuta su siguiente instrucción.

4. Para la espera limitada de los lectores, la cola L de cada semáforo es FIFO.

La sección restante de cualquier proceso no usa los semáforos ni `readcount`, por lo que un proceso que está en ella no puede bloquear a los demás.

4. Demostración

4.1. Lemas

Lema 1 (`mutex` es un candado)**.** Las regiones R1–R4 y R6–R9 son mutuamente excluyentes: nunca hay dos procesos a la vez entre un `wait(mutex)` completado y su `signal(mutex)`.

Demostración. Sea v el valor del semáforo, L su cola de bloqueados y n el número de procesos que ya completaron `wait(mutex)` y todavía no ejecutan `signal(mutex)`. En la implementación del material, si $v < 0$ entonces $|L| = -v$ (un `wait` que deja $v < 0$ encola al proceso y un `signal` con $v \leq 0$ desencola a uno), así que $v + |L| \geq 0$. Se prueba la invariante $v + n + |L| = 1$:

- Al inicio vale $1 + 0 + 0 = 1$.
- `wait` que no bloquea: v baja 1 y n sube 1. `wait` que bloquea: v baja 1 y $|L|$ sube 1.
- `signal`: el que sale baja n en 1 y sube v en 1; si además despierta a un proceso, $|L|$ baja 1 y n sube 1 (el despertado ya completó su `wait`).

En todos los casos la suma se conserva. Como $v + |L| \geq 0$, resulta $n = 1 - (v + |L|) \leq 1$. □

Lema 2 (significado de `readcount`)**.** En todo momento, `readcount` es igual al número de lectores que ya ejecutaron R2 y todavía no ejecutan R7. En particular `readcount` ≥ 0.

Demostración. `readcount` solo se modifica en R2 y en R7, y ambas instrucciones están dentro de una región protegida por `mutex`; por el Lema 1 nunca se ejecutan dos modificaciones a la vez, así que no hay actualizaciones perdidas (la lectura, la suma y la escritura de `readcount++` no se pueden intercalar con las de otro lector). Cada R2 suma 1 y agrega un lector al conjunto descrito, y cada R7 resta 1 y lo quita. Un lector ejecuta R7 solo después de R2, así que el valor nunca es negativo. □

Lema 3 (el grupo de lectores usa `rw_mutex` como un solo proceso)**.** Los `wait(rw_mutex)` de R3 y los `signal(rw_mutex)` de R8 se ejecutan de manera estrictamente alternada: wait, signal, wait, signal, ... Además:

(a) El `wait` de R3 lo ejecuta únicamente el lector cuyo R2 llevó `readcount` de 0 a 1, y el `signal` de R8 únicamente el lector cuyo R7 lo llevó de 1 a 0.

(b) Ningún otro lector puede modificar `readcount` entre R2 y la terminación de R3 del primer lector, ni entre R7 y la terminación de R8 del último.

Demostración. Por el Lema 2, `readcount` cambia de ± 1 en ± 1 y nunca es negativo; por lo tanto las transiciones $0 \to 1$ y $1 \to 0$ se alternan, comenzando por $0 \to 1$. La condición de R3 se evalúa dentro de la región R1–R4 con el valor recién escrito por R2, así que es verdadera exactamente cuando la transición fue $0 \to 1$; igual para R8 con $1 \to 0$ y la región R6–R9. Esto prueba la alternancia y (a). Como R3 y R8 están dentro de las regiones protegidas, el Lema 1 da (b). □

Sea W el número de escritores que completaron `wait(rw_mutex)` (W1) y todavía no ejecutan `signal(rw_mutex)` (W3), es decir, los que están en W2 o en W3. Sea $g \in \{0, 1\}$ la variable auxiliar que vale 1 desde que el `wait` de R3 de un grupo de lectores termina hasta que el `signal` de R8 de ese mismo grupo termina, y 0 en otro caso. Por el Lema 3, g está bien definida y el grupo de lectores cuenta como un único "poseedor" de `rw_mutex`.

Lema 4 (conservación en `rw_mutex`)**.** $W + g \leq 1$.

Demostración. La invariante del Lema 1 vale para cualquier semáforo inicializado en 1: $v + H + |L| = 1$, donde H es el número de poseedores, es decir, de procesos que completaron un `wait` y todavía no ejecutan el `signal` correspondiente, y $v + |L| \geq 0$. Cada escritor poseedor aporta 1 a H (sus `wait` y `signal` van emparejados en W1 y W3). Por el Lema 3, los `wait` y `signal` de los lectores también están emparejados y alternados, así que el grupo de lectores aporta g. Entonces $H = W + g$ y $W + g = 1 - (v + |L|) \leq 1$. □

Lema 5 (un lector en R5 implica $g = 1$)**.** Si un lector r está en R5, entonces $g = 1$.

Demostración. Sea a el lector cuyo R2 produjo la última transición $0 \to 1$ de `readcount` hasta el R2 de r inclusive (puede ser el propio r). Entre esa transición y el R2 de r no hubo una transición $1 \to 0$ (en tal caso el R2 de r o uno posterior sería una nueva transición $0 \to 1$, contra la elección de a). Mientras a está en R3 conserva `mutex`, así que $r \neq a$ no puede ejecutar R2 hasta que a termine R4, es decir, después de que el `wait` de R3 de a terminó; y si $r = a$, llega a R5 después de R3. Por tanto, cuando r llega a R5 el grupo ya adquirió `rw_mutex` y $g = 1$. La siguiente transición $1 \to 0$ no puede ocurrir antes de que r ejecute R7 (Lema 2: r está contado en `readcount`), y r ejecuta R7 después de R5. Así $g = 1$ durante toda la estancia de r en R5. □

4.2. Exclusión mutua

Teorema 1 (exclusión mutua)**.** En ningún estado alcanzable hay dos escritores en W2, ni un escritor en W2 con un lector en R5.

Demostración. Si un escritor está en W2, entonces completó W1 y no ha llegado a W3, de modo que $W \geq 1$. Por el Lema 4, $W + g \leq 1$, de donde $W = 1$ y $g = 0$.

- $W = 1$ significa que hay exactamente un escritor en W2 o W3: no hay otro escritor en W2 (ME1).

- Si además hubiera un lector en R5, el Lema 5 daría $g = 1$, lo cual contradice $g = 0$ (ME2).

Por tanto ME1 y ME2 se cumplen. □

Además, el algoritmo *sí* permite varios lectores en R5: el único lector de un grupo en pasar por R3 es el primero, y los siguientes incrementan `readcount` sin esperar (ya que `readcount` > 1). Esto es lo que se exige a la solución.

4.3. Progreso

Teorema 2 (progreso)**.** Si ningún proceso está en su sección crítica (W2 o R5) y al menos un proceso está en su código de entrada, entonces, en tiempo finito, algún proceso entra a su sección crítica.

Demostración. Se hacen notar tres hechos. (i) Los procesos en su sección restante no poseen ningún semáforo, por lo que no impiden nada. (ii) Un proceso solo puede quedar bloqueado en `wait(mutex)` (R1, R6) o en `wait(rw_mutex)` (W1, R3); `signal` nunca bloquea. (iii) Un lector que tiene `mutex` solo puede estar bloqueado en R3, porque las demás instrucciones de las regiones R1–R4 y R6–R9 no bloquean. Sea $H = W + g$ y se razona por casos sobre H (que es 0 o 1 por el Lema 4).

Caso $H = 0$**.** Por la invariante $v + H + |L| = 1$ con $v + |L| \geq 0$ se tiene $v = 1$ y $|L| = 0$: `rw_mutex` está libre y nadie espera en su cola. Si hay un escritor en W1, su `wait` no se bloquea y entra a W2. Si hay lectores en entrada, quien tiene `mutex` (si lo hay) no se puede quedar bloqueado en R3, porque `wait(rw_mutex)` pasa directamente, y libera `mutex` en tiempo finito (la región es de instrucciones que no bloquean); lo mismo ocurre con un lector que esté terminando su salida. El primer lector en ejecutar R3 con `readcount` $= 1$ pasa `wait(rw_mutex)`, ejecuta R4 y llega a R5.

Caso $W = 1$**.** Como nadie está en W2, el escritor está en W3 y ejecuta `signal(rw_mutex)` en tiempo finito. Si hay procesos en la cola de `rw_mutex`, se despierta al primero: si es un escritor, queda en W2 (entró a la sección crítica); si es el primer lector de un grupo, termina R3, ejecuta R4 y llega a R5 en tiempo finito. Si la cola está vacía, se pasa al caso $H = 0$.

Caso $g = 1$**.** Como nadie está en R5, los lectores contados en `readcount` están en R3–R4 (ya adquirido), R6 o R7. Si alguno está en R4, llega a R5 y entra. De lo contrario todos están saliendo (R6–R9). En este caso ningún lector está bloqueado en R3: ese `wait` solo lo ejecuta quien lleva `readcount` de 0 a 1, y mientras $g = 1$ eso no puede ocurrir porque entre R7 y R8 del último lector (que es el que libera el grupo) conserva `mutex` (Lema 3(b)). Por lo tanto cada lector con `mutex` lo libera en tiempo finito, todos los lectores contados terminan R7, y el último ejecuta R8, que hace $g = 0$ y despierta, si

los hay, a los procesos en cola, quienes entran como en el caso anterior; si no hay nadie en cola se llega al caso $H = 0$.

En todos los casos algún proceso entra en tiempo finito. □

La demostración incluye la ausencia de interbloqueo: no hay un estado alcanzable en el que todos los procesos estén bloqueados con la sección crítica vacía.

4.4. Espera limitada

Lectores: sí (con semáforos FIFO)

Teorema 3 (lectores)**.** Con semáforos de cola FIFO, un lector que solicita entrar entra a su sección crítica en tiempo finito.

Demostración. Un lector r puede esperar en la cola de `mutex` (R1) o, si es el primero de un grupo, en la cola de `rw_mutex` (R3).

- *Cola de `mutex`.* Delante de r hay a lo más $N_r - 1$ lectores (N_r = número de lectores) y ningún escritor, pues los escritores no usan `mutex`. Cada uno libera `mutex` en tiempo finito salvo, quizá, uno que esté bloqueado en R3; si alguno vuelve a solicitar `mutex`, por FIFO queda formado detrás de r, de modo que cada uno pasa a lo más una vez antes que r.
- *Cola de `rw_mutex`.* Un lector a bloqueado en R3 es el primero de un grupo, de modo que $g = 0$; como hay alguien en la cola de `rw_mutex`, ya tiene poseedor ($H = 1$, Lema 4) y este es un escritor en W2–W3. Por FIFO, delante de a hay a lo más N_w escritores, cada uno con una sección crítica finita, y cada uno pasa una sola vez antes que a porque al terminar vuelve a formarse detrás de a. Los lectores nuevos no se interponen: están formados detrás de a en la cola de `mutex`, que a conserva.

Así a obtiene `rw_mutex` en tiempo finito, libera `mutex` en R4, y el siguiente lector de la cola avanza, y por inducción sobre la posición en la cola, r llega a R5 en tiempo finito. □

Los lectores de un grupo ya activo no esperan por `rw_mutex`: solo por `mutex`, durante un tiempo corto. Si los semáforos despertaran a un proceso cualquiera de la cola (no FIFO), un lector podría ser rebasado indefinidamente; esto lo confirma el verificador (sección 5) con 2 lectores y 2 escritores.

Escritores: no

Teorema 4 (inanición de escritores)**.** El algoritmo no cumple la espera limitada para los escritores: existen ejecuciones válidas, con planificador justo y semáforos FIFO, en las que un escritor que ya solicitó la sección crítica espera indefinidamente mientras otros procesos entran una cantidad ilimitada de veces.

Demostración. Sean dos lectores A y B y un escritor W. Cada lector ejecuta repetidamente la sección crítica y vuelve a entrar:

Paso	**Acción**	`readcount`	`rw_mutex`
1	A entra: R2 deja `readcount`$= 1$ y pasa `wait(rw_mutex)`. A lee.	1	0
2	W ejecuta W1: se bloquea en la cola de `rw_mutex`.	1	-1
3	B entra: R2 deja `readcount`$= 2$; R3 no espera. A y B leen.	2	-1
4	A sale (R7: `readcount`$= 1$, no hay `signal`) y vuelve a entrar (R2: `readcount`$= 2$).	2	-1
5	B sale (`readcount`$= 1$) y vuelve a entrar (`readcount`$= 2$).	2	-1
Se repiten 4 y 5 indefinidamente; `readcount` nunca llega a 0.			

Tabla 1. Ejecución en la que el escritor W nunca entra.

En esta ejecución W está bloqueado, de modo que el planificador no lo está dejando sin ejecutarse: simplemente no se le despierta. En cada ronda, los lectores completan al menos dos secciones críticas completas (todas finitas), así que se cumplen los supuestos. Siempre hay al menos un lector contado en `readcount`, por lo que nunca se ejecuta el `signal(rw_mutex)` de R8 que despertaría a W. El número de lectores que entran después de la solicitud de W no tiene cota. □

La inanición es consecuencia de la propia variante: si "ningún lector esperará a menos que un escritor esté accediendo al recurso", un lector nuevo no puede ser detenido por un escritor que *espera*. Por tanto, cualquier algoritmo que cumpla esa regla sufre este defecto, y la espera limitada para escritores solo se obtiene cambiando el enunciado del problema (por ejemplo, la variante en que un escritor en espera impide que entren nuevos lectores, que a su vez puede hacer que esperen los lectores; ver Courtois et al., 1971; Downey, 2016).

5. Verificación con un verificador de modelos

Como comprobación independiente de la demostración se programó un verificador que explora todas las intercalaciones posibles. Cada línea de las etiquetadas es una acción atómica, salvo `readcount++` y `readcount-`, que se dividen en leer y escribir para mostrar que es `mutex` lo que evita las actualizaciones perdidas. Los semáforos se modelan como en el material (valor, cola, bloqueo y despertar). Se revisó:

- ME1 y ME2 en todos los estados alcanzables, y que `readcount` es siempre igual al número de lectores entre R2 y R7 (Lema 2);
- ausencia de interbloqueo y progreso: desde cualquier estado con sección crítica vacía y algún solicitante, sin nuevas solicitudes, siempre hay avance y no hay ciclos sin entrar a la sección crítica;
- inanición: existencia de un ciclo infinito justo en el que un proceso bloqueado nunca avanza mientras otros entran a su sección crítica (todo proceso habilitado ejecuta pasos en el ciclo, o está bloqueado).

Lect.	Escr.	Estados	ME	Interb.	Progreso	Inan. lect.	Inan. escr.
1	1	63	cumple	no	cumple	no	no
1	2	246	cumple	no	cumple	no	no
2	1	1 326	cumple	no	cumple	no	**sí**
2	2	4 940	cumple	no	cumple	no	**sí**
3	1	47 416	cumple	no	cumple	no	**sí**

Tabla 2. Resultados con semáforos FIFO. La inanición de escritores aparece desde que hay dos lectores que pueden traslaparse.

Resultados adicionales:

- **Control negativo.** Al quitar `wait(mutex)` y `signal(mutex)` del lector (2 lectores y 1 escritor), el verificador encuentra en 12 pasos un estado con un lector y un escritor en su sección crítica a la vez (violación de ME2). Esto confirma que el verificador detecta errores y que `mutex` es necesario.
- **Semáforos no FIFO.** Si `signal` despierta a un proceso cualquiera de la cola, con 2 lectores y 2 escritores también aparecen ciclos de inanición para los lectores; ME y progreso se mantienen. Esto justifica el supuesto 4.
- El tamaño de estas instancias es pequeño: la verificación complementa la demostración, que es la que cubre un número arbitrario de procesos.

6. Conclusiones

Condición	**£Se cumple?**	**Base**
Exclusión mutua	Sí (ME1 y ME2)	Teorema 1: $W + g \leq 1$ y lector en R5 $\Rightarrow g = 1$
Progreso	Sí, sin interbloqueo	Teorema 2: análisis por casos de quién posee `rw_mutex`
Espera limitada, lectores	Sí, con semáforos FIFO	Teorema 3
Espera limitada, escritores	**No**	Teorema 4: inanición si los lectores se traslapan

Tabla 3. Resumen.

El algoritmo del material resuelve correctamente el problema en lo esencial: protege los datos (exclusión mutua) y nunca se detiene (progreso), y permite la máxima concurrencia entre lectores. Su límite es la espera de los escritores, que es una propiedad de la variante con prioridad a lectores y no un defecto de programación. Si la aplicación necesita que los escritores no esperen indefinidamente, hay que usar la variante con prioridad a escritores o una versión que impida el traslape continuo de lectores, aceptando el costo en concurrencia.

Referencias

Courtois, P. J., Heymans, F., & Parnas, D. L. (1971). Concurrent control with "readers" and "writers". *Communications of the ACM, 14*(10), 667–668.

Downey, A. B. (2016). *The little book of semaphores* (2.ł ed.). Green Tea Press.

Rodríguez Úbeda, D. J. (2026). *Coordinación y sincronización de procesos: Secciones críticas* [Presentación de diapositivas, Unidad IVa]. Sistemas Operativos, Universidad de Sonora.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ł ed.). Wiley.